Write a program in C/C++/Java to simulate the Banker's algorithm for deadlock avoidance. Consider at least 3 processes in the system, with 4 resource classes having at least one resource instance for each class. Assume the values for Available, Allocation, MAX, and request from a particular process from your side. Program must reflect two cases, where a safe sequence exists for one and safe sequence does not exist for another.

```
#include <iostream>
#include <vector>

using namespace std;

const int NUM_PROCESSES = 3;
const int NUM_RESOURCES = 4;

bool isSafe(const vector<int> &available, const vector<vector<int>> &max, const vector<vector<int>> &allocation, vector<vector<int>> &need) {
   vector<int> work = available;
   vector<bool> finish(NUM_PROCESSES, false);
   vector<int> safeSequence;

   for (int count = 0; count < NUM_PROCESSES; ++count) {
      bool foundProcess = false;
      for (int i = 0; i < NUM_PROCESSES; i++) {
         if (!finish[i]) {
            bool canExecute = true;
            for (int j = 0; j < NUM_RESOURCES; j++) {
               if (need[i][j] > work[j]) {
                  canExecute = false;
                  break;
               }
            }
            if (canExecute) {
               for (int k = 0; k < NUM_RESOURCES; k++)
```

```
                work[k] += allocation[i][k];
            safeSequence.push_back(i);
            finish[i] = true;
            foundProcess = true;
          }
        }
      }
      if (!foundProcess) return false;
    }

    cout << "System is in a SAFE state.\nSafe sequence: ";
    for (int i : safeSequence) cout << "P" << i << " ";
    cout << endl;

    return true;
}

vector<vector<int>> calculateNeed(const vector<vector<int>> &max, const vector<vector<int>>
&allocation) {
    vector<vector<int>> need(NUM_PROCESSES, vector<int>(NUM_RESOURCES));
    for (int i = 0; i < NUM_PROCESSES; i++)
        for (int j = 0; j < NUM_RESOURCES; j++)
            need[i][j] = max[i][j] - allocation[i][j];
    return need;
}

void requestResources(vector<int> &available, vector<vector<int>> &max, vector<vector<int>>
&allocation, int processId, vector<int> &request) {
    vector<vector<int>> need = calculateNeed(max, allocation);

    // Check if request can be granted
    for (int j = 0; j < NUM_RESOURCES; j++) {
```

```
        if (request[j] > need[processId][j]) {
            cout << "Error: Process has exceeded its maximum claim.\n";
            return;
        }
        if (request[j] > available[j]) {
            cout << "Error: Resources are not available.\n";
            return;
        }
    }

    // Temporarily allocate resources
    for (int j = 0; j < NUM_RESOURCES; j++) {
        available[j] -= request[j];
        allocation[processId][j] += request[j];
        need[processId][j] -= request[j];
    }

    // Check if system is in safe state
    if (isSafe(available, max, allocation, need)) {
        cout << "Resources have been allocated to process " << processId << " safely.\n";
    } else {
        // Rollback allocation
        for (int j = 0; j < NUM_RESOURCES; j++) {
            available[j] += request[j];
            allocation[processId][j] -= request[j];
            need[processId][j] += request[j];
        }
        cout << "Resources cannot be allocated safely. Rolling back.\n";
    }
}
```

```
void request_safe_case() {
    vector<int> available = {3, 2, 1, 1};
    vector<vector<int>> max = {
        {2, 1, 1, 3},
        {1, 1, 2, 1},
        {3, 1, 1, 1}
    };
    vector<vector<int>> allocation = {
        {1, 0, 0, 1},
        {0, 1, 1, 0},
        {1, 0, 0, 1}
    };
    vector<vector<int>> need = calculateNeed(max, allocation);

    cout << "\n--- Safe Case ---" << endl;
    isSafe(available, max, allocation, need);

    vector<int> request = {1, 0, 0, 0}; // Example request for safe case
    requestResources(available, max, allocation, 0, request);
}

void request_unsafe_case() {
    vector<int> available = {1, 1, 0, 1};
    vector<vector<int>> max = {
        {3, 1, 1, 3},
        {1, 1, 2, 2},
        {4, 1, 1, 1}
    };
    vector<vector<int>> allocation = {
        {2, 0, 0, 1},
        {1, 1, 1, 1},
```

```
        {1, 0, 0, 0}
    };
    vector<vector<int>> need = calculateNeed(max, allocation);

    cout << "\n--- Unsafe Case ---" << endl;
    isSafe(available, max, allocation, need);

    vector<int> request = {1, 1, 1, 0}; // Example request for unsafe case
    requestResources(available, max, allocation, 0, request);
}

int main() {
    request_safe_case();
    request_unsafe_case();
    return 0;
}
```